

# rtmle for diabetes register data

From continuous-time histories to regime-specific risk

Funding: NNF23OC0084961

Thomas Alexander Gerds

Kathrine Kold Sørensen

Christian Tobias Torp-Pedersen

Alessandra Meddis

Anders Munch

Johan Sebastian Ohlendorff

Helene Charlotte Wiese Rytgaard

Joshua Schwab

Maya Petersen

Mark van der Laan

Zeyi Wang

Andrew Mertens

Nerissa Nance

# One sentence

The R-package `rtmle` implements targeted minimum loss-based estimation for longitudinal observational or randomized event-history data<sup>1</sup>.

It is designed around an analysis object that stores

- the time grid, variable names, hyper parameters
- long or wide data
- treatment regimes and targets
- nuisance-model formulas and fitted objects
- estimates, influence curves, and bootstrap

```
devtools::install_github("tagteam/rtmle")
```

---

<sup>1</sup>Related packages: `ltmle`, `lmtp`, `gformula`, `tmle3`

## A diabetes research question

Imagine a target trial among people with type 2 diabetes:

Compare the 60-month risk of MACE under regimens/policies that continue GLP1, continue SGLT2, continue both, or continue neither.

The four risks are causal parameters such as

$$\begin{aligned}\psi_g(P) &= P\{\text{MACE}^a(60) = 1\}, \\ a &\in \{\text{GLP1-only, SGLT2-only, both, neither}\}.\end{aligned}$$

This is an illustration of the software, not a recommendation of how to design or analyze data.

## Why the software needs our attention

The clinical question is short. The data, the design and the estimation algorithm are not:

- observations occur at irregular person-specific times
- treatment can change several times between target-trial visits
- outcome, death, and loss to follow-up have different meanings
- the expected time to onset of a treatment effect depends on both the drug and the outcome under consideration
- “take GLP1” is incomplete unless we say what happens when side-effects become severe
- every longitudinal TMLE needs many treatment, censoring, and outcome models.

The package makes these choices executable and inspectable.

Data are collected in continuous time

## A synthetic diabetes register cohort

The function `simulate_cohort` produces an event-history table. Here the variables are named for the diabetes application:

```
diabetes_parameters <- list(  
  intercept_age = 60, var_age = 10, intercept_sex = 1,  
  intercept_HbA1c = 58, var_HbA1c = 8,  
  intercept_GLP1 = 1, intercept_SGLT2 = 1,  
  intercept_deltaHbA1c = 0, var_deltaHbA1c = 2,  
  scale_MACE = .0015, scale_death = .0007, scale_dropout = .0012,  
  scale_af = 0.01, scale_hypoglycemia = .0015,  
  effect_age_MACE = .01, effect_HbA1c_MACE = .02,  
  effect_GLP1_MACE = -.3, effect_SGLT2_MACE = -.3,  
  effect_GLP1_death = -.1, effect_SGLT2_death = -.1,  
  effect_GLP1_dropout = .1, effect_SGLT2_dropout = .1,  
  effect_GLP1_hypoglycemia = .3, effect_hypoglycemia_GLP1 = -10,  
  effect_SGLT2_hypoglycemia = .2, effect_hypoglycemia_SGLT2 = -3,  
  effect_GLP1_deltaHbA1c = -1.5,  
  effect_SGLT2_deltaHbA1c = -1.2,  
  effect_age_GLP1 = .005, effect_HbA1c_GLP1 = .01,  
  effect_age_SGLT2 = .005, effect_HbA1c_SGLT2 = .01  
)
```

## Generate a cohort (lava engine)

```
diabetes_cohort <- simulate_cohort(  
  n = 1000, seed = 20260915,  
  # clock is in months  
  max_follow = 60,  
  # baseline  
  baseline_variables = list(age = "normal", sex = "binomial", HbA1c  
    = "normal"),  
  baseline_visit = list(  
    GLP1 = "binomial", SGLT2 = "binomial"  
  ),  
  # hooks make the function flexible  
  post_baseline_visit_hook = function(X) {  
    X[, ']:='(GLP1 = rbinom(.N, 1, .45), SGLT2 = rbinom(.N, 1, .30))]  
  },  
  # visit schedule  
  visit_schedule = list(  
    mean = 6, sd = .2, skip = 0,  
    minimum_time_between_visits = 1  
  ),
```

# Generate a cohort

```
# blood test
visit_measurements = list(deltaHbA1c = "normal"),
# treatment
visit_events = list(
  GLP1 = "binomial", SGLT2 = "binomial"
),
# comorbidity and side effects
intermediate_events = list(
  af = "Weibull",
  hypoglycemia = "Weibull"
),
# end-of-story
absorbing_events = list(
  MACE = "Weibull",
  death = "Weibull",
  dropout = "Weibull"
),
# regression parameters
parameter_values = diabetes_parameters
)
```

# Simulated cohort

```
diabetes_cohort
```

|       | id    | time      | event        | age      | sex   | HbA1c    | af    | hypoglycemia |
|-------|-------|-----------|--------------|----------|-------|----------|-------|--------------|
|       | <int> | <num>     | <fctr>       | <num>    | <num> | <num>    | <num> | <num>        |
| 1:    | 1     | 0.000000  | baseline     | 40.66673 | 1     | 49.12022 | 0     | 0            |
| 2:    | 1     | 5.727589  | visit        | 40.66673 | 1     | 49.12022 | 0     | 0            |
| 3:    | 1     | 11.583591 | visit        | 40.66673 | 1     | 49.12022 | 0     | 0            |
| 4:    | 1     | 16.716125 | af           | 40.66673 | 1     | 49.12022 | 1     | 0            |
| 5:    | 1     | 22.431758 | hypoglycemia | 40.66673 | 1     | 49.12022 | 1     | 1            |
| ---   |       |           |              |          |       |          |       |              |
| 5747: | 999   | 0.000000  | baseline     | 76.65837 | 0     | 64.79631 | 0     | 0            |
| 5748: | 999   | 4.214845  | af           | 76.65837 | 0     | 64.79631 | 1     | 0            |
| 5749: | 999   | 9.242383  | dropout      | 76.65837 | 0     | 64.79631 | 1     | 0            |
| 5750: | 1000  | 0.000000  | baseline     | 58.49227 | 0     | 64.19030 | 0     | 0            |
| 5751: | 1000  | 5.525743  | MACE         | 58.49227 | 0     | 64.19030 | 0     | 0            |

```
rf <- register_format(  
  diabetes_cohort,  
  treatment_variables = c("GLP1", "SGLT2")  
)
```

# What the register data format looks like

```
names(rf)
```

```
[1] "baseline_data" "timevar_data"
```

```
head(rf$baseline_data)
```

Key: <id>

|    | id    | age      | sex   | HbA1c    |
|----|-------|----------|-------|----------|
|    | <int> | <num>    | <num> | <num>    |
| 1: | 1     | 40.66673 | 1     | 49.12022 |
| 2: | 2     | 42.07977 | 1     | 67.20141 |
| 3: | 3     | 51.42940 | 1     | 56.22718 |
| 4: | 4     | 51.90558 | 1     | 48.56435 |
| 5: | 5     | 32.28980 | 0     | 57.48259 |
| 6: | 6     | 77.13011 | 1     | 48.14418 |

```
names(rf$timevar_data)
```

```
[1] "GLP1"          "SGLT2"          "deltaHbA1c"     "af"              "hypoglycemia"
[8] "dropout"
```

# What the register data format looks like

```
head(rf$timevar_data$MACE,3)
```

|    | id    | date      |
|----|-------|-----------|
|    | <int> | <num>     |
| 1: | 2     | 20.895953 |
| 2: | 3     | 14.029143 |
| 3: | 5     | 4.715178  |

```
head(rf$timevar_data$GLP1,3)
```

|    | id    | start_date | end_date |
|----|-------|------------|----------|
|    | <int> | <num>      | <num>    |
| 1: | 1     | 5.495158   | 19.46030 |
| 2: | 3     | 10.072385  | 14.02914 |
| 3: | 4     | 6.004910   | 60.00000 |

```
head(rf$timevar_data$deltaHbA1c,5)
```

Key: <id, date>

|    | id    | date      | value      |
|----|-------|-----------|------------|
|    | <int> | <num>     | <num>      |
| 1: | 1     | 0.000000  | 0.0000000  |
| 2: | 1     | 5.495158  | 0.5202452  |
| 3: | 1     | 11.519541 | -5.0657637 |
| 4: | 1     | 17.167420 | -2.9206167 |
| 5: | 2     | 0.000000  | 0.0000000  |

rtmle

# Initialize the analysis object

```
x <- rtmle_init(  
  time_unit = "Months since treatment start",  
  time_grid = seq(0, 60, 6),  
  name_id = "id",  
  name_outcome = "MACE",  
  name_competing = "death",  
  name_censoring = "dropout",  
  censored_levels = c("uncensored", "censored"),  
  censored_label = "censored",  
  # hyperparameters  
  minority_threshold = 8,  
  weight_truncation = c(0, 1),  
  prediction_range = c(0.0001, 0.9999)  
)
```

The object stores two synchronized clocks:

- `time_grid_scale = (0,6,12,...,60)`, the time scale used by the data
- `time_grid = (0,1,...,10)`, the interval index used internally.

There are ten intervention nodes,  $(0, \dots, 9)$ , and ten outcome intervals,  $(1, \dots, 10)$ .

# The initialized object

```
print(x)
```

Targeted minimum loss-based analysis of longitudinal data on a discretized time

Package version: 2026.9.10

Subject id variable: id

Outcome variable: MACE

Censoring variable: dropout

Competing risk: death

Intervention nodes: 0, 1, ..., 9

Minority threshold: 8

Weight truncation: (0,1)

Prediction range: (0.0001,0.9999)

TODO: The object contains no baseline\_data yet. Add them with the function `rtml`

TODO: The object contains no time varying data yet. Add them with `rtmle::add_wi`

TODO: The object contains no outcome data yet. Add them with `rtmle::add_wide_da`

TODO: The object contains no regimes. Add them with the function `'protocol'`.

Prepared data: n=0, p=-1

TODO: Use the function `'model_formula'` to initialize the formula for the nuisance

TODO: Use the function `'run_rtmle'` to estimate the nuisance parameter models an

Diagnostics: No warnings (yet)

## Add data, then choose a mapping

```
# add data to the object
x <- add_baseline_data(
  x,
  data = rf$baseline_data
)
x <- add_long_data(
  x,
  outcome_data = rf$timevar_data$MACE,
  censored_data = rf$timevar_data$dropout,
  competing_data = rf$timevar_data$death,
  timevar_data = rf$timevar_data[c(
    "hypoglycemia",
    "af",
    "deltaHbA1c",
    "GLP1",
    "SGLT2"
  )]
)
```

The long data are not yet the data on which the nuisance models are fit. That conversion is performed by `discretize_data ...`

# Map events and measurements

```
x <- discretize_data(  
  x,  
  # date format: provide a person-specific start date  
  start_followup_date = 0,  
  hypoglycemia = list(method = "event"),  
  af = list(method = "event"),  
  deltaHbA1c = list(method = "locf"),  
  GLP1 = list(  
    variable = "GLP1",  
    method = "exposure_percent",  
    threshold = 0.8  
  ),  
  SGLT2 = list(  
    variable = "SGLT2",  
    method = "exposure_percent",  
    threshold = 0.8  
  )  
)
```

Here the treatment definition is binary: taking value 1 in an interval if the exposure covers more than 80% of that interval.

## The mapping dictionary (to be expanded)

| method(s)        | format                                                       | meaning on grid    |
|------------------|--------------------------------------------------------------|--------------------|
| event            | date                                                         | ever               |
| event_interval   | date                                                         | in interval        |
| locf             | date, value                                                  | carried forward    |
| measurement      | date, value                                                  | aggregated         |
| any_exposure     | start <sub>date</sub> , end <sub>date</sub>                  | any overlap        |
| has_exposure     | start <sub>date</sub> , end <sub>date</sub>                  | fraction threshold |
| exposure_time    | start <sub>date</sub> , end <sub>date</sub>                  | duration exposed   |
| exposure_percent | start <sub>date</sub> , end <sub>date</sub>                  | fraction exposed   |
| custom           | date, start <sub>date</sub> ,<br>end <sub>date</sub> , value | user defined       |

The discretization is necessary for the discrete time analysis and part of the *scientific definition* of covariates and treatment and not just a data-cleaning convenience.

# Prepare the handoff to run\_rtmle

```
x <- prepare_rtmle_data(x)
x$prepared_data
```

Key: <id>

|       | id    | age      | sex   | HbA1c    | hypoglycemia_0 | deltaHbA1c_0 |
|-------|-------|----------|-------|----------|----------------|--------------|
|       | <int> | <num>    | <num> | <num>    | <num>          | <num>        |
| 1:    | 1     | 40.66673 | 1     | 49.12022 | 0              | 0            |
| 2:    | 2     | 42.07977 | 1     | 67.20141 | 0              | 0            |
| 3:    | 3     | 51.42940 | 1     | 56.22718 | 0              | 0            |
| 4:    | 4     | 51.90558 | 1     | 48.56435 | 0              | 0            |
| 5:    | 5     | 32.28980 | 0     | 57.48259 | 0              | 0            |
| ---   |       |          |       |          |                |              |
| 996:  | 996   | 49.10872 | 1     | 56.52349 | 0              | 0            |
| 997:  | 997   | 48.51101 | 1     | 45.33607 | 0              | 0            |
| 998:  | 998   | 70.35807 | 1     | 60.24374 | 0              | 0            |
| 999:  | 999   | 76.65837 | 0     | 64.79631 | 0              | 0            |
| 1000: | 1000  | 58.49227 | 0     | 64.19030 | 0              | 0            |

|  | GLP1_0 | SGLT2_0 | dropout_1 | MACE_1 | death_1 | hypoglycemia_1 |
|--|--------|---------|-----------|--------|---------|----------------|
|--|--------|---------|-----------|--------|---------|----------------|

|    | <num> | <num> | <fctr>     | <num> | <num> | <num> |
|----|-------|-------|------------|-------|-------|-------|
| 1: | 1     | 0     | uncensored | 0     | 0     | 0     |
| 2: | 0     | 1     | uncensored | 1     | 0     | NA    |
| 3: | 1     | 1     | uncensored | 0     | 0     | 0     |
| 4: | 1     | 0     | uncensored | 0     | 0     | 0     |
| 5: | 0     | 1     | uncensored | 0     | 0     | 0     |

Protocolled treatment regimes

# The regimes are part of the estimand

The treatment table is a regime for the target trial dictating what the treatment should be at each intervention node.

```
x <- protocol(  
  x, name = "always_GLP1_never_SGLT2",  
  intervention = data.frame(  
    time_node=x$intervention_nodes,  
    "GLP1" = factor(rep("1",10),levels = c("0","1")),  
    "SGLT2" = factor(rep("0",10),levels = c("0","1"))  
  )  
)  
# short form  
x <- protocol(  
  x, name = "always_SGLT2_never_GLP1",  
  intervention = c(1,0),  
  treatment_variables = c("SGLT2", "GLP1")  
)
```

# Intervention table

```
x$regimes$always_SGLT2_never_GLP1$intervention_table
```

|     | time_node | variable | value  |
|-----|-----------|----------|--------|
|     | <int>     | <char>   | <fctr> |
| 1:  | 0         | SGLT2_0  | 1      |
| 2:  | 1         | SGLT2_1  | 1      |
| 3:  | 2         | SGLT2_2  | 1      |
| 4:  | 3         | SGLT2_3  | 1      |
| 5:  | 4         | SGLT2_4  | 1      |
| 6:  | 5         | SGLT2_5  | 1      |
| 7:  | 6         | SGLT2_6  | 1      |
| 8:  | 7         | SGLT2_7  | 1      |
| 9:  | 8         | SGLT2_8  | 1      |
| 10: | 9         | SGLT2_9  | 1      |
| 11: | 0         | GLP1_0   | 0      |
| 12: | 1         | GLP1_1   | 0      |
| 13: | 2         | GLP1_2   | 0      |
| 14: | 3         | GLP1_3   | 0      |
| 15: | 4         | GLP1_4   | 0      |
| 16: | 5         | GLP1_5   | 0      |
| 17: | 6         | GLP1_6   | 0      |
| 18: | 7         | GLP1_7   | 0      |
| 19: | 8         | GLP1_8   | 0      |
| 20: | 9         | GLP1_9   | 0      |

# Visualize adherence to protocolled regimes

```
plot_adherence(x)+theme(legend.position="top")
```



## Intervene? Yes, but not always

```
# Partial control: NA means 'no intervention'
x <- protocol(
  x, name = "limit_GLP1_allow_SGLT2",
  intervention = data.frame(
    time_node=x$intervention_nodes,
    "GLP1" = factor(c(rep("1",5),rep(NA,5)),levels = c("0","1")),
    "SGLT2" = factor(c(0,0,rep(NA,8)),levels = c("0","1"))
  ),
  # static rule
  intervene_function = "intervene"
)
```

## History-dependent, dynamic rules

A custom intervention function specifies a permanent stop of treatment in case of a contra-indication:

```
x <- protocol(  
  x, name = "treat-until-hypo",  
  intervention = data.frame(  
    time_node=x$intervention_nodes,  
    "GLP1" = factor(rep("1",10),levels = c("0","1")),  
    "SGLT2" = factor(rep("0",10),levels = c("0","1"))  
  ),  
  intervene_function = "stop_after_hypo"  
)
```

# Stop treatment in case of a medical contradiction

```
stop_after_hypo <- function(data, intervention_table, time_node){  
  # apply the underlying static regime  
  # which sets treatment to 1  
  out <- intervene(data, intervention_table, time_node)  
  for (i in which(!is.na(intervention_table$value) &  
                  intervention_table$time_node <= time_node &  
                  grepl("^GLP1_", intervention_table$variable))) {  
    v <- intervention_table$variable[i]  
    # find history of the hypoglycemia process  
    h <- intersect(paste0("hypoglycemia_",  
                          0:intervention_table$time_node[i]),  
                  names(data))  
    # change the value to zero for people  
    # with a history of hypoglycemia  
    if (length(h))  
      out[[v]][rowSums(do.call(cbind, lapply(h, function(z)  
        data[[z]] %in% c(1, "1")))) > 0] <-  
        factor("0", levels = levels(out[[v]]))  
  }  
  out  
}
```

too complicated? rtmle is user-friendly

```
x <- protocol(  
  x, name = "treat_until_hypo",  
  intervention = data.frame(  
    time_node=x$intervention_nodes,  
    "GLP1" = factor(rep("1",10),levels = c("0","1")),  
    "SGLT2" = factor(rep("0",10),levels = c("0","1"))  
  ),  
  intervene_function = treat_unless(  
    contra_indication = "hypoglycemia",  
    action = c("0",NA)  
  )  
)
```

# Adherence table

```
summary_adherence(  
  x,  
  regimes = c("always_GLP1_never_SGLT2", "treat_until_hypo")  
)
```

|     | regime<br><char>        | time_node<br><int> | n_initiated<br><int> | n_adherent<br><int> | n_outcome<br><int> | n_death<br><int> |
|-----|-------------------------|--------------------|----------------------|---------------------|--------------------|------------------|
| 1:  | always_GLP1_never_SGLT2 | 0                  | 313                  | 313                 | 52                 | 8                |
| 2:  | always_GLP1_never_SGLT2 | 1                  | 313                  | 241                 | 39                 | 6                |
| 3:  | always_GLP1_never_SGLT2 | 2                  | 313                  | 66                  | 30                 | 7                |
| 4:  | always_GLP1_never_SGLT2 | 3                  | 313                  | 15                  | 20                 | 5                |
| 5:  | always_GLP1_never_SGLT2 | 4                  | 313                  | 1                   | 24                 | 1                |
| 6:  | always_GLP1_never_SGLT2 | 5                  | 313                  | 0                   | 13                 | 1                |
| 7:  | always_GLP1_never_SGLT2 | 6                  | 313                  | 0                   | 4                  | 1                |
| 8:  | always_GLP1_never_SGLT2 | 7                  | 313                  | 0                   | 11                 | 1                |
| 9:  | always_GLP1_never_SGLT2 | 8                  | 313                  | 0                   | 5                  | 1                |
| 10: | always_GLP1_never_SGLT2 | 9                  | 313                  | 0                   | 4                  | 0                |
| 11: | treat_until_hypo        | 0                  | 313                  | 313                 | 52                 | 8                |
| 12: | treat_until_hypo        | 1                  | 313                  | 241                 | 39                 | 6                |
| 13: | treat_until_hypo        | 2                  | 313                  | 70                  | 30                 | 7                |
| 14: | treat_until_hypo        | 3                  | 313                  | 19                  | 20                 | 5                |
| 15: | treat_until_hypo        | 4                  | 313                  | 6                   | 24                 | 1                |
| 16: | treat_until_hypo        | 5                  | 313                  | 4                   | 13                 | 1                |
| 17: | treat_until_hypo        | 6                  | 313                  | 3                   | 4                  | 1                |
| 18: | treat_until_hypo        | 7                  | 313                  | 3                   | 11                 | 1                |
| 19: | treat_until_hypo        | 8                  | 313                  | 1                   | 5                  | 1                |
| 20: | treat_until_hypo        | 9                  | 313                  | 0                   | 4                  | 0                |

## Multiple treatments: propensity factorization

```
x <- protocol(  
  x, name = "always_GLP1_and_SGLT2",  
  intervention = data.frame(  
    time_node = x$intervention_nodes,  
    GLP1 = factor(rep("1", 10), levels = c("0", "1")),  
    SGLT2 = factor(rep("1", 10), levels = c("0", "1"))  
  ),  
  multiple_treatment_factorization = "sequential"  
)
```

| factorization | treatment factorization                             |
|---------------|-----------------------------------------------------|
| "joint"       | one model for $(GLP1_k, SGLT2_k) \mid H_k$          |
| "sequential"  | $GLP1_k \mid H_k$ , then $SGLT2_k \mid GLP1_k, H_k$ |
| "independent" | separate $GLP1_k \mid H_k$ and $SGLT2_k \mid H_k$   |

The choice changes the estimated intervention probabilities and therefore the clever covariates. It is not merely a computational switch.

## Handle dynamic adherence probabilities

For a rule such as “stop GLP1 after hypoglycemia”, the function `protocol` has the argument `adherence_model_strata`:

```
x <- protocol(  
  x, name = "GLP1-until-hypo",  
  intervention = data.frame(  
    time_node=x$intervention_nodes,  
    "GLP1" = factor(rep("1",10),levels = c("0","1")),  
    "SGLT2" = factor(rep("0",10),levels = c("0","1"))  
  ),  
  intervene_function = treat_unless(  
    contra_indication = "hypoglycemia",  
    action = c("0",NA)  
  ),  
  adherence_model_strata = "hypoglycemia"  
)
```

This has the effect that different models are fitted in subgroups defined by hypoglycemia to predict adherence to the regime.

If a stratum contains (almost) only adherent observations, `rtmle::fitter` predicts a constant probability.

## Estimands are defined as targets

```
x <- target(x, name = "ateMACE", estimator = "tmle",  
            regimes = c("always_GLP1_never_SGLT2",  
                        "always_SGLT2_never_GLP1"))
```

The object may handle multiple targets sequentially

```
x <- target(x, name = "suppl_MACE_risk", estimator = "tmle",  
            regimes = c("always_GLP1_allow_SGLT2",  
                        "treat_until_hypo"))
```

Super learning

# Specifying variables for the nuisance parameter models

```
x <- model_formula(x)
x$models$time_1$always_GLP1_allow_SGLT2
x$models$time_3$outcome
x$models$time_8$censoring
```

```
$'GLP1_1,SGLT2_1'
```

```
$'GLP1_1,SGLT2_1'$formula
```

```
[1] "I(1*(GLP1_1==1&SGLT2_1==0)) ~ age + sex + HbA1c + GLP1_0 + SGLT2_0"
```

```
$MACE_4
```

```
$MACE_4$formula
```

```
[1] "I(1*(MACE_4==1)) ~ age + sex + HbA1c + hypoglycemia_1 + hypoglycemia_2 + h
+ af_1 + af_2 + af_3 + deltaHbA1c_1 + deltaHbA1c_2 + deltaHbA1c_3 + GLP1_0 + GL
+ GLP1_3 + SGLT2_0 + SGLT2_1 + SGLT2_2 + SGLT2_3"
```

```
$dropout_9
```

```
$dropout_9$formula
```

```
[1] "I(1*(dropout_9=='uncensored')) ~ age + sex + HbA1c + hypoglycemia_1 + hypo
hypoglycemia_3 + hypoglycemia_4 + hypoglycemia_5 + hypoglycemia_6 + hypoglycemi
+ af_1 + af_2 + af_3 + af_4 + af_5 + af_6 + af_7 + af_8 + deltaHbA1c_1 + deltaH
+ deltaHbA1c_4 + deltaHbA1c_5 + deltaHbA1c_6 + deltaHbA1c_7 + deltaHbA1c_8 + GL
+ GLP1_3 + GLP1_4 + GLP1_5 + GLP1_6 + GLP1_7 + GLP1_8 + SGLT2_0 + SGLT2_1 + SGL
+ SGLT2_5 + SGLT2_6 + SGLT2_7 + SGLT2_8"
```

# Specifying variables for the nuisance parameter models

```
x <- model_formula(x,exclusion_rules = list("GLP1_*" = "SGLT2_0"),
  Markov = c("af","hypoglycemia","deltaHbA1c"))
x$models$time_1$always_GLP1_allow_SGLT2
x$models$time_3$outcome
x$models$time_8$censoring
```

```
$'GLP1_1,SGLT2_1'
```

```
$'GLP1_1,SGLT2_1'$formula
```

```
[1] "I(1*(GLP1_1==1&SGLT2_1==0)) ~ age + sex + HbA1c + GLP1_0"
```

```
$MACE_4
```

```
$MACE_4$formula
```

```
[1] "I(1*(MACE_4==1)) ~ age + sex + HbA1c + GLP1_0 + GLP1_1 + GLP1_2 + GLP1_3 + SGLT2_0 + SGLT2_1 + SGLT2_2 + SGLT2_3 + hypoglycemia_3 + af_3 + deltaHbA1c_3"
```

```
$dropout_9
```

```
$dropout_9$formula
```

```
[1] "I(1*(dropout_9=='uncensored')) ~ age + sex + HbA1c + GLP1_0 + GLP1_1 + GLP1_4 + GLP1_5 + GLP1_6 + GLP1_7 + GLP1_8 + SGLT2_0 + SGLT2_1 + SGLT2_2 + SGLT2_5 + SGLT2_6 + SGLT2_7 + SGLT2_8 + hypoglycemia_8 + af_8 + deltaHbA1c_8"
```

## What is being learned?

For each time node and protocolled regime, `run_rtmle` fits nuisance models for:

- treatment or treatment-combination probabilities
- remaining uncensored probabilities
- the outcome regression in the backward sequential regression.

The outcome regression is fitted with `rtmle_predicted_outcome` as a continuous predicted-probability outcome from the previous step

$$\hat{Q}_{k+1} \longrightarrow \text{TMLE fluctuation at } k \longrightarrow \hat{Q}_k.$$

The hyperparameter `weight_truncation` is applied to the cumulative intervention probabilities, the hyperparameter `prediction_range` is applied to the predicted probability outcomes before the TMLE update step.

## Built-in single learners (to be expanded)

| learner                    | role          | controls                                                                               |
|----------------------------|---------------|----------------------------------------------------------------------------------------|
| <code>learn_glm</code>     | glm           | <code>formula</code> , <code>vars</code> , <code>maxit</code>                          |
| <code>learn_glmnet</code>  | elastic-net   | <code>alpha</code> , <code>lambda</code> , <code>selector</code> , <code>nfolds</code> |
| <code>learn_ranger</code>  | forest        | <code>num.trees</code> , <code>min.node.size</code> , ...                              |
| <code>learn_xgboost</code> | boosted trees | many tuning arguments                                                                  |

For `learn_glmnet`, `selector = "undersmooth"` uses the least penalized value that still fits; `"min"` and `"1se"` use cross-validated choices.

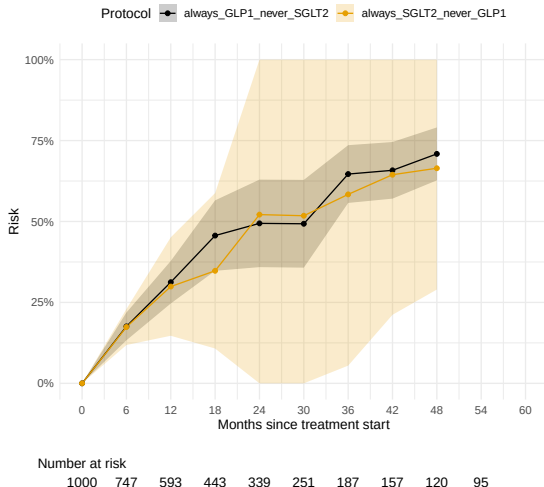
# Running the LTMLE algorithm

```
x <- run_rtmle(
  x,
  time_horizon = 4,
  learner = "learn_glmnet",
  target = "ateMACE"
)
summary(x)
```

|    | Target<br><fctr>  | Regime<br><fctr>        | Reference<br><char>     | Target_parameter<br><fctr> | Tim            |                            |
|----|-------------------|-------------------------|-------------------------|----------------------------|----------------|----------------------------|
| 1: | ateMACE           | always_GLP1_never_SGLT2 |                         | Risk                       |                |                            |
| 2: | ateMACE           | always_SGLT2_never_GLP1 |                         | Risk                       |                |                            |
| 3: | ateMACE           | always_SGLT2_never_GLP1 | always_GLP1_never_SGLT2 | Risk_difference            |                |                            |
| 4: | ateMACE           | always_SGLT2_never_GLP1 | always_GLP1_never_SGLT2 | Risk_ratio                 |                |                            |
|    | Estimate<br><num> | P_value<br><num>        | Standard_error<br><num> | Lower<br><num>             | Upper<br><num> | Estimate (CI_95)<br><char> |
| 1: | 0.49418460        | 1.0000000               | 0.06897885              | 0.3589885                  | 0.6293807      | 49.4 [35.9;62.9]           |
| 2: | 0.52127256        | 1.0000000               | 0.36544868              | 0.0000000                  | 1.0000000      | 52.1 [0.0;100.0]           |
| 3: | 0.02708795        | 0.9419384               | 0.37191469              | -0.7018514                 | 0.7560273      | 2.7 [-70.2;75.6]           |
| 4: | 1.05481343        | 0.9404934               | 0.71485670              | 0.2598295                  | 4.2821589      | 1.1 [0.3;4.3]              |

# Estimating ATE at multiple time horizons

```
x <- run_rtmle(x,refit = FALSE,time_horizon = 1:8,learner = "learn_glmnet", target = "ateMACE")  
plot(x,targets = "ateMACE")+theme(legend.position="top")
```



## Calling a super learner

```
x <- run_rtmle(  
  x,  
  learner = list(  
    folds = 10,  
    ensemble_method = "ipa",  
    learners = list(  
      "glmnet",  
      "elastic"=list(fun="learn_glmnet",alpha=0.5,selector="min"),  
      "forest"=list(fun="learn_ranger",num.trees=50)  
    )  
  ),  
  time_horizon = 4,  
  seed = 20260915  
)
```

Each inner list names a learner function and passes the remaining entries as arguments.

## The three ensemble methods (to be expanded)

| method   | weight rule                           |
|----------|---------------------------------------|
| discrete | one candidate: minimum Brier score    |
| nnls     | non-negative weights, normalized      |
| ipa      | Brier improvement over the null model |

Models worse than the null model receive zero ipa weight. If every candidate receives zero weight, the null model prediction is used.

The `fitter` checks if outcome variables have sufficiently many different values, controlled by `minority_threshold`, if it doesn't simply predicts the average value for all subjects.

The number of folds for super learner cross-validation is another hyperparameter.

`superlearn` checks if all levels of all categorical predictors are present in all folds.

## Details that matter

- The same library can be used for treatment, censoring, and outcome models, but the formulas and the subjects “at-risk” differ by node.
- The sequential predicted outcome is continuous, so candidate learners must be able to predict both binary outcomes and probabilities passed backward.
- Constant outcomes, constant predictors, and missing factor levels are recorded under `x$diagnostics`.
- Stored censoring and propensity of treatment models are not refitted unless explicitly requested.

Diagnostics

# Diagnostics

```
print(x)
```

Targeted minimum loss-based analysis of longitudinal data on a discretized time scale.

```
Package version:      2026.9.10
Subject id variable:  id
Outcome variable:     MACE
Censoring variable:   dropout
Competing risk:       death
Intervention nodes:   0, 1, ..., 9
Minority threshold:   8
Weight truncation:    (0,1)
Prediction range:     (0.0001,0.9999)
Baseline data:        n=1000, p=3
Timevar data:         hypoglycemia, af, deltaHbA1c, GLP1, SGLT2
Outcome data:         n=1000, p=MACE_0, MACE_1, ..., death_10
Regimes:              always_GLP1_never_SGLT2, always_GLP1_allow_SGLT2
Target:               MACE_risk [always_GLP1_never_SGLT2, always_GLP1_allow_SGLT2]
Number of models:     40
Learner:              SuperLearner (10-fold CV)
                      +learn_glmnet (alpha=0.5, selector="min")
                      +learn_ranger (num.trees=50)
Fitted time horizons: 4, 1, ..., 8
Diagnostics:
                      constant_outcome_variables (n=4)
                      no_positive_weights (n=2)
                      superlearner_null_model (n=12)
```

## Coefficients from fitted nuisance models

The exported function `plot_model_coefficients.` returns both the log-odds ratios (only `glm` and `glmnet`) and a `ggplot` object:

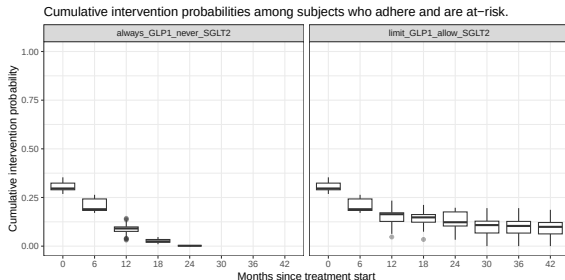
```
coef(x)
```

|      | time   |                         | node           |  | outcome |                | terms       |  | beta         |
|------|--------|-------------------------|----------------|--|---------|----------------|-------------|--|--------------|
|      | <char> |                         | <char>         |  | <char>  |                | <char>      |  | <num>        |
| 1:   | time_0 | always_GLP1_never_SGLT2 | GLP1_0,SGLT2_0 |  |         |                | (Intercept) |  | 0.000000000  |
| 2:   | time_0 | always_GLP1_never_SGLT2 | GLP1_0,SGLT2_0 |  |         |                | age         |  | -0.003513968 |
| 3:   | time_0 | always_GLP1_never_SGLT2 | GLP1_0,SGLT2_0 |  |         |                | sex         |  | -0.191999230 |
| 4:   | time_0 | always_GLP1_never_SGLT2 | GLP1_0,SGLT2_0 |  |         |                | HbA1c       |  | -0.002836185 |
| 5:   | time_0 |                         | outcome        |  | MACE_1  |                | (Intercept) |  | 0.000000000  |
| ---  |        |                         |                |  |         |                |             |  |              |
| 220: | time_7 |                         | outcome        |  | MACE_8  |                | SGLT2_61    |  | 1.635276701  |
| 221: | time_7 |                         | outcome        |  | MACE_8  |                | SGLT2_71    |  | -1.534662120 |
| 222: | time_7 |                         | outcome        |  | MACE_8  | hypoglycemia_7 |             |  | 0.603971784  |
| 223: | time_7 |                         | outcome        |  | MACE_8  |                | af_7        |  | -1.377461591 |
| 224: | time_7 |                         | outcome        |  | MACE_8  | deltaHbA1c_7   |             |  | 0.000000000  |

```
plot_model_coefficients(x, regime = "always_GLP1_allow_SGLT2")
```

# The cumulative intervention-probability plot

```
plot_IPW(x, regimes = c("limit_GLP1_allow_SGLT2", "always_GLP1_never_SGLT2"))
```



These are the probabilities that enter the cumulative inverse-probability weights, shown only among subjects who are adherent and still at risk.

# Weight truncation

```
run_rtmle(x,time_horizon=4,weight_truncation=c(0.02,0.98),targets =  
  "ateMACE")
```

```
Estimate (CI_95)  
      <char>
```

```
1: 50.4 [37.3;63.5]  
2: 51.2 [29.0;73.3]  
3: 0.7 [-25.0;26.5]  
4: 1.0 [0.6;1.7]
```

```
run_rtmle(x,time_horizon=4,weight_truncation=c(0,1),targets = "  
  ateMACE")
```

```
Estimate (CI_95)  
      <char>
```

```
1: 49.4 [35.9;62.9]  
2: 52.3 [0.0;100.0]  
3: 2.9 [-70.4;76.1]  
4: 1.1 [0.3;4.3]
```

## Four checks for a defensible analysis

1. **Design:** population, time zero, follow-up, outcome, competing event, and censoring are explicit.
2. **Mapping:** each long-format variable has a documented grid method and baseline convention.
3. **Regimes:** the treatment rule, multiple-treatment factorization, and support are inspectable.
4. **Learning:** learner library, folds, selectors, diagnostics, and any weight truncation are recorded with the result.
5. **Inference** cheap bootstrap sub-sampling confidence intervals

Make the estimand executable.

Development phase extensions for `rtmle`:

- Calibrated estimands via  $\alpha$ -scaling (Rytgaard et al.)
- Sequential regression in continuous time (Ohlendorff et al.)
- Multi-state model estimands (Munch et al.)